Project Overview: Verifiable Agentic Optimization

Internal research brief

April 2026

The problem

Many AI agents are now evaluated in settings where they do not simply answer a question once. They modify something, run a test, observe the result, and try again. Examples include editing code, optimizing a training script, repairing a proof, changing a query engine, or improving a data-processing workflow.

Current benchmarks usually reduce this process to a final score: did the agent succeed, what was its best score, or how long did it take to pass a threshold? Those numbers are useful, but they hide the mechanism. Two agents can reach the same final score for very different reasons. One may know the right general strategy but write poor local code. Another may write strong code but repeatedly choose the wrong kind of intervention. A third may improve only after seeing feedback from the verifier.

This project starts from a simple claim: for iterative, verifiable tasks, the trajectory is not a byproduct. It is the main scientific object. The sequence of edits, verifier outputs, mode choices, costs, and revisions can tell us why a model succeeds or fails, and therefore what kind of post-training would help.

The concrete task

The experimental task is open-ended optimization of a stateful query engine. The editable artifact is a Python file called `solution.py`. It implements a `CandidateQueryEngine` with methods such as `put`, `delete`, `get`, `range_sum`, `aggregate_count`, and `top_k`.

An agent starts from a simple baseline implementation and tries to improve it. After each proposed edit, an external verifier checks two things: first, whether the implementation is exactly correct against a trusted reference; second, whether it is faster or more memory-efficient on generated workload traces. Incorrect implementations fail; correct ones receive a scalar loss based mostly on latency and partly on memory.

The key experimental choice is that every step is structured around six editing modes:

Mode	Meaning
`layout`	change the primary data representation
`indexing`	improve range-query access paths
`topk`	improve the top-k selection algorithm
`caching`	add or revise query-result memoization
`summaries`	add maintained aggregation structures
`micro`	make constant-factor code optimizations

At each step, the model gives a probability distribution over these six modes and produces one candidate edit for each mode. All six edits are evaluated by the verifier. However, in the first experimental phase, the model only sees the result of the edit it believed was most promising. This keeps the online interaction realistic while still giving the researchers counterfactual information about what would have happened under the other modes.

What we measure

The project separates four quantities that are normally tangled together in the final score.

Routing mismatch. Did the model put probability mass on the editing modes that were actually productive? If it believed caching was best but indexing produced the strongest verified improvement, that is routing error.

Within-mode competence. If a mode is fixed, how good is the model at writing the corresponding patch? This distinguishes choosing the right kind of action from executing it well.

Feedback use. Does verifier evidence improve the model's next distribution over modes? A good model should become not only more confident after feedback, but more aligned with the modes that verification shows to be productive.

Cost per step. Some models may perform well but spend much more time, money, or tokens. Cost is part of the task, not an afterthought.

The endpoint metrics remain important: best verified score within budget and success rate above a chosen threshold. The difference is that these endpoints are now accompanied by a mechanistic diagnosis.

How the experiments will work

The first experiments use three tiers of models under the same single-agent scaffold: a strong closed-source teacher, a mid-tier closed-source model, and a weaker open-weight code model that can be post-trained. The planned roles are simple: the strong model provides an upper bound and possible teacher signal; the mid-tier model checks whether diagnostic quantities scale with capability; the open-weight model is the student to be improved.

The models are compared on the same workload profiles. A profile is a named configuration of the workload generator: seeds, trace families, trace lengths, key-space size, repetitions, and related parameters. Some profiles are used for development; held-out profiles are reserved for final evaluation.

The first post-training experiment focuses only on routing. This is the cleanest first target because the verifier itself can tell us which mode was productive at each step. The student is trained to predict the productive mode distribution from the current checkpoint and history. We then evaluate whether this targeted routing training improves held-out performance more than generic fine-tuning with the same amount of data.

What success would look like

The strongest result would show three things.

First, different models should not merely have different endpoint scores; they should have different diagnostic signatures. A weak model may have high routing mismatch, low within-mode competence, poor feedback use,

or some combination of these.

Second, the diagnostic should predict the right intervention. If the weak model's dominant deficit is routing, routing-specific post-training should improve it more than an undiagnosed generic training baseline.

Third, improvements should transfer to held-out workload profiles. This matters because the goal is not to overfit a single query-engine configuration, but to demonstrate that trajectory diagnostics can guide model improvement in a principled way.

Why it matters

The broader idea is that a benchmark can be more than a leaderboard. If a task is designed so that its trajectories are observable and verifiable, it can become a diagnostic instrument. It can tell us not only which model is better, but what kind of training data a weaker model needs.

That shift is important for agentic AI. As agents become more autonomous, endpoint scores alone will become less informative. We will need methods that reveal whether an agent is choosing the right strategy, using feedback correctly, and executing local changes competently. This project proposes one concrete, testable way to do that.